Arduino Corridor Cleaner — Full Build Guide (Vacuum + Mop + Obstacle Avoidance + Clap Control + Zoning + Auto-Return)

> **Scope:** Small, course-demo robot designed for school corridor cleaning. Selects **vacuum** on dry floor and **mop** on wet floor using **metal-contact wet sensor**. Adds **ultrasonic obstacle avoidance**, **clap-based start/stop**, **basic zoning/coverage mapping**, and **battery auto-return to dock** via a **tape guide (line follow)**.

---

1) System Overview

Core controller: Arduino Uno (ATmega328P)

Functional blocks - **Mobility:** 2 × DC gear motors + caster, driven by TB6612FNG (or L298N if already available). - **Vacuum:** 12 V mini centrifugal blower, MOSFET-switched. - **Mopping:** Mop pad on a hinged arm (SG90 servo), 6–12 V mini diaphragm/peristaltic pump to wet mop (MOSFET-switched) + small water tank. - **Wet/Dry decision:** Stainless metal contacts under front bumper; AC-excited sensing to reduce corrosion. - **Obstacle avoidance:** HC-SR04 ultrasonic on a small pan servo (SG90) for left-center-right scan. - **Sound control:** Electret mic module with comparator (LM393 board or MAX9814 mic amp). Detect double-clap pattern. - **Zoning / basic mapping:** Wheel encoders + IMU (MPU-6050) for dead-reckoning; simple grid coverage (boustrophedon stripes) with corridor boundaries. - **Auto-return to dock:** Low-battery detection via voltage divider; follow a **black tape guide** using 2× TCRT5000 line sensors back to a docking station; spring contacts for charging.

Top-level state machine

```
IDLE ↔ WAIT_FOR_CLAP
  │ (start)
  ▼
EXPLORE/CLEAN ──► AVOID_OBSTACLE ──► resume path
  │                    │
  ├── (wet) → CLEAN_WET (mop+pump)
  └── (dry) → CLEAN_DRY (vacuum)
  │
  └── (low battery) → RETURN_HOME (find tape) → DOCKING → CHARGING → IDLE
```

---

2) Bill of Materials (BOM)

Control & Power

- 1× **Arduino Uno** (or Nano to save space)
- 1× **Motor driver** TB6612FNG (preferred) *or* L298N
- 1× **Buck converter** (LM2596) 12 V → 5 V for logic/servos
- 1× **Battery pack**: 2S Li-ion (2×18650, 7.4 V nominal) *or* 3S (11.1 V) depending on blower; include a BMS board (2S/3S) rated ≥10 A
- 1× **Charger module** at dock: TP5100 (for 2S) or appropriate multi-cell charger
- 1× **Fuse** (e.g., 10 A inline) + reverse-polarity protection (Schottky diode or ideal diode board)
- 2× **Power MOSFET** logic-level (e.g., IRLZ44N/IRLZ34N) + flyback diode for pump (1N5819/SS34) and blower (if brushed)
- 1× **Power switch**, wiring, terminal blocks

Mobility & Sensing

- 2× **12 V DC gear motors** with wheels (with built-in encoders *preferred*), 1× caster
- 1× **HC-SR04 ultrasonic**
- 2× **SG90 servos** (one for ultrasonic pan, one for mop arm)
- 2× **TCRT5000 line sensors** (for tape guide to dock)
- 1× **MPU-6050** IMU (I^2C)
- 2× **Hall sensors** (A3144) + 2× magnets *if motors lack encoders*
- 1× **Electret microphone sensor**: LM393 sound sensor *or* MAX9814 mic amp module (better SNR)
- **Metal wet contacts**: 2× stainless screws/plates + mounting hardware; 10 kΩ and 100 kΩ resistors; 1× 1 µF cap; optional 74HC14 Schmitt trigger

Cleaning Hardware

- 1× **Mini centrifugal blower** (12 V) + duct to suction inlet
- 1× **Mini diaphragm or peristaltic pump** (6–12 V) + silicone tubing
- 1× **Water tank** (100–250 ml) + valve/check-valve
- 1× **Mop pad** (microfiber) + hinged arm/bracket
- 1× **Debris bin** with mesh filter

Dock & Chassis

- **Dock base** with **black tape** path from corridor to dock
- 2× **Spring contacts** (pogo pins or copper pads) on robot and matching on dock
- 1× **12 V DC adapter** for dock (current ≥ blower + pump + charge overhead as needed)
- **Chassis** plate, standoffs, screws, cable ties; splash guards

Optional/UX

- 1× **0.96" OLED (I^2C)** for status
- LEDs/Buzzer for feedback

---

3) Power & Wiring Architecture

Suggested rails (example with 3S pack): - **Battery (11.1–12.6 V)** → - **Buck** to **5 V** for Arduino + sensors + servos - **Direct** to blower (if 12 V) through MOSFET - **Direct** to pump (6–12 V) through MOSFET (use buck to 9 V if your pump is 6–9 V)

Grounds: All module grounds **must share a common GND**.

Battery sensing: Voltage divider to Arduino A0 (e.g., 100 kΩ top, 33 kΩ bottom for $\leq$ 16.8 V full-scale). Add 0.1 µF to A0-GND.

Flyback protection: Pump and any brushed motors need diodes across coils. The blower often contains a motor; add diode if brushed.

Typical pin map (adjust as you like): - **Motors (TB6612):** - AIN1 D7, AIN2 D8, PWMA D5 (ENA) - BIN1 D12, BIN2 D13, PWMB D6 (ENB) - STBY D4 - **Encoders:** - Left: D2 (INT0), D3 (INT1) - Right: A2, A3 (if simple single-channel) - **Ultrasonic (HC-SR04):** TRIG D9, ECHO D10 - **Pan servo:** D11 (Servo) - **Mop arm servo:** D3 (or any PWM pin; juggle with encoders as needed) - **Sound sensor:** A1 (analog) or D14/A0 if digital comparator output - **Wet sensor contacts:** Drive pin DA7 (A7 as digital), Sense pin A6 (analog) *UNO analog-only pins A6/A7 are input-only; use D7.. if you need drive; see circuit below* - **Line sensors:** A4, A5 (analog) - **IMU (MPU-6050):** I^2C on A4 (SDA), A5 (SCL) — if these are used for line sensors, move line sensors to A1/A2 - **OLED:** I^2C (shared) - **Blower MOSFET gate:** DA2 (or any free digital) - **Pump MOSFET gate:** DA1 (or any free digital)

> Reconcile pins based on your exact shields/modules. Ensure timer/PWM availability for motor speed control.

---

4) Metal-Contact Wet Sensor (Corrosion-Aware Design)

Goal: Decide **wet vs dry** at the leading edge of the robot.

Why AC excitation? DC across contacts causes electrolysis/corrosion. We'll alternately drive and sense to pseudo-AC excite.

Simple robust approach: - **Contacts:** Two stainless steel screws ~5–8 mm apart, 2–3 mm above floor, mounted on an insulating bracket. Add a thin sponge/guard to prevent shorting by metal debris. - **Circuit:** 1. **Drive** one contact with a 1 kHz square wave from Arduino (digital pin through 1 kΩ resistor). 2. **Sense** the other contact on an **analog input** through a 100 kΩ resistor + 1 µF to ground (forms an envelope detector). 3. Optional **74HC14** after a voltage divider for a clean digital edge. - **Firmware:** - Toggle the drive pin HIGH/LOW every 0.5 ms. - Sample the analog sense; high readings indicate conduction (wet). - Use a moving average and hysteresis thresholds (e.g., `WET_ON > 250`, `WET_OFF < 200` on 10-bit ADC) to avoid chatter.

Alternative super-simple (more corrosion): Use internal pull-up on sense pin; set drive pin LOW; when water bridges, the sense pin reads LOW. If you choose this, pulse the drive (duty < 10%) to reduce corrosion.

5) Obstacle Avoidance

- Mount **HC-SR04** on **pan servo** (±45°). Each cycle: look **center**, **left**, **right** and take median of 3 pings at each angle.
- If **distance <** `D_STOP` (e.g., 20 cm), stop → back up 10 cm → rotate toward open direction.
- Add a short **blind-time** after a turn to avoid re-triggering on the same obstacle.

---

6) Sound-Triggered Control (Clap Toggle)

- Use **MAX9814** module (preferred) to get a stable envelope; or LM393 sound sensor.
- Detect **two claps** within **500–800 ms** windows:
- Threshold by dynamic baseline (median filter) + fixed offset.
- Debounce each clap (ignore events for 150 ms after a peak).
- On valid double-clap: toggle IDLE ↔ CLEAN.

---

7) Vacuum vs Mop Actuation Logic

- **Dry (not wet):** Enable **blower**, lift mop arm (servo to UP), pump OFF.
- **Wet:** Disable blower, lower mop arm (servo to DOWN), **pulse the pump** periodically (e.g., 200 ms ON every 2–3 s) to dampen pad but avoid puddles.
- Add a **post-wet drying pass**: after leaving a wet patch, keep mop down for extra 20 cm then lift.

---

8) Zoning & Basic Mapping (Corridor-Friendly)

Assumptions: Corridor ~ long rectangle. Demo uses **dead-reckoning** (encoders + IMU yaw) and a simple **grid**.

- **Grid:** 10 cm cells; e.g., 20×5 grid stored as bytes (visited flags) → ~100 B.
- **Coverage pattern: Boustrophedon** (lawnmower): sweep forward along the corridor length; at end or obstacle, shift sideways by 1 strip (corridor width/`n_stripes`), then sweep back.
- **Strip alignment:** Use IMU yaw to maintain heading; re-zero yaw after each turn. Encoders estimate distance.
- **Zones:** Divide the corridor into named segments (Z1, Z2, ...) by distance from dock (e.g., every 3 m). Store current `zone_id` for UI/logs.
- **Persistence:** For a demo, map may be RAM-only. Optionally store coverage percentage in EEPROM every minute.

  This is *not* SLAM. It's deterministic coverage using odometry suitable for a course demo.

---

9) Auto-Return & Docking

Low-battery detection: - Compute pack voltage from divider; compare to thresholds (e.g., 2S: `LOW=7.2 V`, `CRITICAL=7.0 V`). Add 100–200 ms RC filtering + software averaging.

Return strategy (simple & robust): - **Find guide tape:** On low battery, rotate in place while reading 2× TCRT5000; when both see black, align and **line-follow** back to dock. - **Docking:** Slow approach (PWM ~30%), keep center to tape. When front bump/limit switch (optional) or **charging voltage detected**, stop motors → **CHARGING** state.

Dock electronics: - Dock has **12 V adapter → charger module (TP5100 for 2S)** → spring contacts. - Robot's pack connects through BMS to those contacts. Add LED indicators on dock for charging/full.

> If you cannot lay tape in the corridor, add a cheap **IR beacon** at the dock and do rotational homing (but tape is far simpler for a demo).

---

10) Software Architecture

Recommended libraries: - `AccelStepper` (if you ever use steppers) / or simple PWM for DC - `Servo.h` - `NewPing.h` (or your own HC-SR04 timing) - `Wire.h` + MPU-6050 helper - Optional: `EEPROM.h`, small `PID` lib for heading hold

Core modules: - `Motion`: set speeds, turns, distance using encoders + yaw hold - `Sensors`: ultrasonic scan, wet sensor filter, line sensors, sound detect, battery voltage - `Actuators`: blower, pump, mop servo, pan servo - `CoveragePlanner`: boustrophedon strips across grid - `FSM`: handles states & transitions

State summary: - `IDLE`, `WAIT_FOR_CLAP`, `CLEAN_DRY`, `CLEAN_WET`, `AVOID_OBSTACLE`, `RETURN_HOME`, `DOCKING`, `CHARGING`

---

11) Key Circuits (Text Description)

A) Wet sensor (AC-excited): - Arduino D7 → 1 kΩ → **Contact A** - **Contact B** → 100 kΩ → A0 (sense) - A0 → 1 µF → GND - Add 220 kΩ from A0→GND as a bleed if needed - Firmware toggles D7 at ~1 kHz

B) Battery monitor: - Pack + → 100 kΩ → A1 → 33 kΩ → GND, 0.1 µF A1→GND - Calibrate with multimeter

C) MOSFET driver (blower/pump): - Arduino D8 (gate) → 100 Ω → MOSFET G - 10 kΩ from G→GND - MOSFET D to motor −; motor + to battery + (or buck output) - MOSFET S to GND - Flyback diode across motor

D) Line sensors: - TCRT5000 analog outputs to A2/A3; power from 5 V; small trimmer sets threshold if using digital modules

E) Encoders: - If present, connect to D2/D3 (interrupts). If absent, add hall + magnets on wheel hubs.

---

12) Tuning Values (Starting Points)

- Ultrasonic thresholds: `D_STOP=20 cm`, `D_TURN=30 cm`
- Wet sensor ADC: `WET_ON=250`, `WET_OFF=200` (10-bit scale)
- Line sensor: black tape threshold ~ `< 400` on 10-bit ADC (depends on module/height)
- Clap detection: envelope threshold = `baseline + 80` (tune), double-clap window `600 ms`
- Mop servo angles: UP = 30°, DOWN = 120° (depends on linkage)
- Pump pulse: 200 ms ON every 2–3 s while moving on wet

---

13) Pseudocode (Skeleton)

```
setup() {
  initPins(); initMotors(); initServos(); initIMU(); initADC();
  calibrateLine(); calibrateWet(); baselineSound();
  state = IDLE;
}

loop() {
  readAllSensors();
  updateWetFiltered();
  updateBattery();
  if (lowBattery() && state != RETURN_HOME && state != DOCKING && state !=
CHARGING) state = RETURN_HOME;

  switch(state) {
    case IDLE:
      stopAll(); blowerOff(); pumpOff(); mopUp();
      if (doubleClapDetected()) state = CLEAN_DRY; // will switch internally
based on wet
      break;

    case CLEAN_DRY:
      blowerOn(); pumpOff(); mopUp();
      followCoverageStrip();
      if (wetDetected()) state = CLEAN_WET;
      if (obstacleAhead()) state = AVOID_OBSTACLE;
      break;

    case CLEAN_WET:
      blowerOff(); mopDown(); pumpPulse();
      followCoverageStrip();
```

```
      if (!wetDetected()) { postWetTimer(); if (timerDone()) mopUp(); state =
CLEAN_DRY; }
      if (obstacleAhead()) state = AVOID_OBSTACLE;
      break;

    case AVOID_OBSTACLE:
      stopAll(); backUp(10); turnTowardOpen();
      state = (wetDetected() ? CLEAN_WET : CLEAN_DRY);
      break;

    case RETURN_HOME:
      searchForTape();
      if (tapeFound()) { lineFollowToDock(); state = DOCKING; }
      break;

    case DOCKING:
      approachSlow();
      if (chargingDetected()) { stopAll(); state = CHARGING; }
      else if (timeout) { // couldn't dock
        state = RETURN_HOME; }
      break;

    case CHARGING:
      if (packFull()) state = IDLE;
      break;
  }
}
```

Helper: wet detection (AC-excited)

```
// called in timer ISR or fast loop
void driveWetPin() { digitalWrite(WET_DRIVE_PIN, !digitalRead(WET_DRIVE_PIN)); }

void updateWetFiltered() {
  int raw = analogRead(WET_SENSE_PIN);
  wetEMA = (wetEMA*7 + raw)/8; // simple low-pass
  if (!wetState && wetEMA > WET_ON) wetState = true;
  if (wetState && wetEMA < WET_OFF) wetState = false;
}
```

Helper: double-clap

```
void updateSound() {
  int v = analogRead(SOUND_PIN);
  baseline = 0.99*baseline + 0.01*v;
```

```
  if (v > (baseline + THRESH)) { if (millis()-lastClap>150) {claps++;
lastClap=millis();} }
  if (claps==2 && (millis()-lastClap)<800) { toggleRequested=true; claps=0; }
  if ((millis()-lastClap)>800) claps=0;
}
```

Helper: coverage

```
void followCoverageStrip() {
  holdHeading(yawTarget);
  setSpeed(BASE_SPEED);
  if (distanceAlongStrip() > STRIP_LEN || endDetected()) {
    stopAll(); shiftSideways(STRIP_SPACING); yawTarget =
normalize(yawTarget+180);
    resetDistance();
  }
}
```

---

14) Build Steps (Step-by-Step)

1. **Chassis & drivetrain**: Mount motors+wheels, caster, battery bay, electronics plate. Ensure ground clearance for mop.
2. **Vacuum module**: Fit blower and duct; add debris bin/filter; confirm airflow.
3. **Mop assembly**: Hinge + servo horn; test UP/DOWN angles; mount pad; route tubing from pump and tank to pad.
4. **Sensors**: Install ultrasonic on pan servo (front top), line sensors near floor (front underside), metal contacts (front edge), mic module (away from motors), IMU (center of mass).
5. **Power wiring**: Battery → fuse/BMS → buck (5 V) → Arduino/servos/sensors; MOSFETs for blower/pump; common ground.
6. **Dock**: Mount spring contacts, charger board, and lay black tape path. Verify polarity before first dock.
7. **Firmware bring-up**:
8. Blink test; read battery voltage and calibrate divisor.
9. Read IMU yaw; confirm stable heading.
10. Test encoders & straight-line driving.
11. Test ultrasonic pings and pan scan.
12. Test wet sensor raw values: dry vs damp cloth; tune `WET_ON/OFF`.
13. Test mop servo limits; pump pulse.
14. Test blower MOSFET.
15. Test line sensors over tape; tune thresholds.
16. Test clap detection and toggle.
17. **Integrate FSM**: Start with `CLEAN_DRY` only → add wet logic → add avoidance → add return/docking.

18. **Field trial**: In a 2–3 m mock corridor, confirm coverage stripes, wet/dry switching, and docking.

---

15) Safety & Practical Notes

- **Water + electronics:** Use splash guards; keep 5 V electronics elevated. Conformal coat exposed PCBs near floor.
- **Current draw:** Blower can spike several amps—size MOSFETs, wiring, and fuse appropriately.
- **Electrolysis:** Prefer AC-style excitation for wet sensor; rinse contacts after salty spills.
- **Servos:** Avoid stalling (set mechanical end-stops). Provide dedicated 5 V rail with enough current ($\geq$1 A burst per servo).
- **Battery care:** Never charge Li-ion without a proper charger/BMS. Verify dock polarity.
- **Noise:** Clap control may falsely trigger in loud environments; consider long-press button as secondary start/stop.

---

16) Minimal Parts Substitution Guide

- No encoders? Use **time-based dead-reckoning** (less accurate) + more frequent wall/tape corrections.
- No IMU? Use straight-line correction by comparing left/right encoders only.
- No docking charger? Still implement **return to tape** and stop near dock marker; charge manually.

---

17) Test Checklist

- [ ] Battery reading matches multimeter (±0.1 V)
- [ ] Wet sensor flips reliably with damp cloth; dry recovers within 2 s
- [ ] Mop down only on wet; blower never runs when mop is down
- [ ] Avoidance: stop <20 cm; recover path
- [ ] Line follow stable at 0.2–0.3 m/s
- [ ] Clap toggle responds to 2 claps, ignores single clap
- [ ] Dock contact: LED on charger shows charging

---

18) What to Hand In (Course Demo)

- Short video showing: start via clap → dry vacuum → transition to wet area → mopping → obstacle avoidance → low-battery simulated → auto-return and dock.
- Wiring diagram (block + key connections)
- Code (FSM + modules) with comments
- Tuning notes and limitations (what you'd add next: better mapping, IR beacon homing, brush module, better pump control)

Appendix A — Quick Dry/Wet Decision Table

Floor state	Wet contacts	Blower	Mop arm	Pump
Dry	LOW conduction	ON	UP	OFF
Damp	MED conduction	OFF	DOWN	Pulsed
Wet/puddle	HIGH conduction	OFF	DOWN	OFF (avoid flooding)

Appendix B — Corridor Coverage Basics

- Choose strip spacing slightly **less** than mop/brush width to avoid gaps.
- Re-zero yaw each 180° turn to limit drift.
- Mark corridor ends (optional tape markers) to detect strip boundaries reliably.

You can adapt pin mappings or modules you already have; the architecture remains the same. If you want, I can turn this into a starter **Arduino sketch** tailored to your exact pin choices and modules.